



RO:BIT
ROBOT SPORT GAME TEAM

Team. RO : BIT | **신입생 교육
Python 2일차**

RO : BIT 16th
지능형로봇팀
백인엽



INDEX

에러와 디버깅

예외 처리

파일 입출력과 데이터 검증

CSV 심화

JSON 직렬화

모듈·패키지·의존성

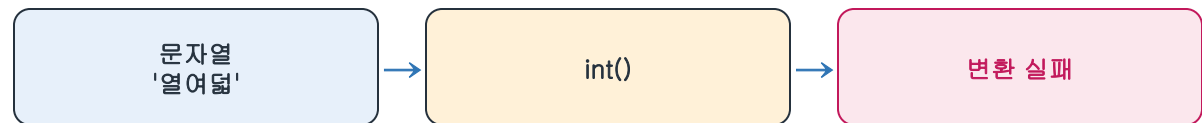
NumPy와 통합 실습

에러와 디버깅

-ValueError

자료형은 맞지만 그 값의 내용이 변환 규칙에 맞지 않을 때 발생한다

```
text = "열여덟"  
age = int(text)  
print(age)
```



메시지에서 찾을 단서

- ① 끝줄의 ValueError가 오류 종류를 알려준다
- ② 따옴표 안 '열여덟'이 변환에 실패한 실제 값이다
- ③ int()는 숫자 모양의 문자열만 정수로 바꾼다
- ④ 고치기: '18'을 사용하거나 isdigit()로 먼저 확인한다

```
(venv) PS C:\robit\python> python main.py  
Traceback (most recent call last):  
  File "main.py", line 2, in <module>  
    age = int(text)  
ValueError: invalid literal for int() with base 10: '열여덟'
```



에러와 디버깅

-IndexError

리스트·튜플 같은 순서 자료에서 존재하지 않는 위치를 조회할 때 발생한다

```
scores = [85, 90, 95]  
print(scores[3])
```



메시지에서 찾을 단서

- ① list index out of range는 범위를 벗어났다는 뜻이다
- ② 원소가 3개이면 사용할 수 있는 인덱스는 0, 1, 2다
- ③ len(scores)는 3이지만 마지막 인덱스는 2다
- ④ 고치기: scores[-1] 또는 index < len(scores)를 사용한다

```
(venv) PS C:\robit\python> python main.py  
Traceback (most recent call last):  
  File "main.py", line 2, in <module>  
    print(scores[3])  
IndexError: list index out of range
```



에러와 디버깅

-SyntaxError

파이썬 문법 규칙에 맞지 않아 프로그램을 시작하기 전에 발견되는 오류다

```
score = 80
if score >= 60
    print("합격")
```

if score >= 60
콜론 누락



if score >= 60:
문법 완성

메시지에서 찾을 단서

- ① SyntaxError는 실행 중이 아니라 코드를 읽는 단계에서 발생한다
- ② ^ 표시는 파이썬이 문법 문제를 감지한 위치 근처다
- ③ if 조건문 끝에는 콜론(:)이 필요하다
- ④ 고치기: if score >= 60: 처럼 문장을 완성한다

```
(venv) PS C:\robit\python> python main.py
File "main.py", line 2
    if score >= 60
        ^
SyntaxError: expected ':'
```



에러와 디버깅

-NameError

아직 정의하지 않았거나 철자가 다른 변수·함수 이름을 사용할 때 발생한다

```
price = 1200
total = price * quantity
print(total)
```



메시지에서 찾을 단서

- ① name 'quantity'가 문제 이름을 그대로 보여준다
- ② 해당 줄보다 앞에서 quantity가 만들어졌는지 확인한다
- ③ 대소문자와 철자도 서로 다른 이름으로 처리된다
- ④ 고치기: quantity = 3처럼 사용 전에 값을 정의한다

```
(venv) PS C:\robit\python> python main.py
Traceback (most recent call last):
  File "main.py", line 2, in <module>
    total = price * quantity
NameError: name 'quantity' is not defined
```

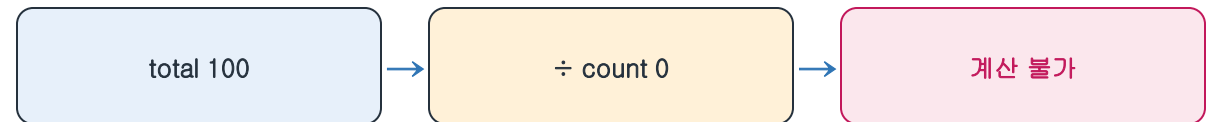


에러와 디버깅

-ZeroDivisionError

나눗셈의 분모가 0이 되어 계산할 수 없을 때 발생한다

```
total = 100  
count = 0  
average = total / count
```



메시지에서 찾을 단서

- ① division by zero는 분모가 0이라는 뜻이다
- ② 오류가 난 식에서 / 또는 //, % 연산을 찾는다
- ③ 입력이 없거나 필터 결과가 0개일 때 자주 발생한다
- ④ 고치기: `count == 0`을 먼저 처리한 뒤 나눈다

```
(venv) PS C:\robit\python> python main.py  
Traceback (most recent call last):  
  File "main.py", line 3, in <module>  
    average = total / count  
ZeroDivisionError: division by zero
```

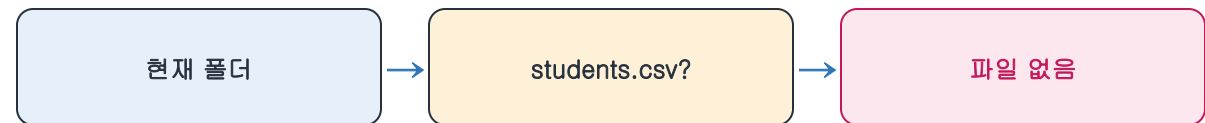


에러와 디버깅

-FileNotFoundError

open()에 전달한 파일이 현재 작업 폴더나 지정 경로에 없을 때 발생한다

```
path = "students.csv"
with open(path) as file:
    print(file.readline())
```



메시지에서 찾을 단서

- ① No such file or directory 뒤의 파일명을 확인한다
- ② 상대 경로는 현재 작업 폴더를 기준으로 해석된다
- ③ 파일명·확장자·폴더 위치가 실제와 같은지 확인한다
- ④ 고치기: Path.exists()로 확인하거나 올바른 경로를 지정한다

```
(venv) PS C:\robit\python> python main.py
Traceback (most recent call last):
  File "main.py", line 2, in <module>
    with open(path) as file:
FileNotFoundError: [Errno 2] No such file or directory: 'students.csv'
```



에러와 디버깅

-TypeError

연산이나 함수가 기대한 자료형과 실제 값의 자료형이 다를 때 발생한다

```
age = 20
print("나이: " + age)
```

str
'나이: '

+ ×

int
20

메시지에서 찾을 단서

- ① str과 int처럼 충돌한 두 자료형을 메시지에서 찾는다
- ② type(age)를 출력하면 실제 자료형을 확인할 수 있다
- ③ 문자열 + 정수는 자동으로 변환되지 않는다
- ④ 고치기: f"나이: {age}" 또는 str(age)를 사용한다

```
(venv) PS C:\robit\python> python main.py
Traceback (most recent call last):
  File "main.py", line 2, in <module>
    print("나이: " + age)
TypeError: can only concatenate str (not "int") to str
```



에러와 디버깅

-AttributeError

객체가 가지고 있지 않은 속성이나 메서드를 호출할 때 발생한다

```
name = "Python"  
name.append(" 3")
```



메시지에서 찾을 단서

- ① 'str' object는 현재 객체의 자료형을 알려준다
- ② no attribute 'append'는 없는 메서드 이름을 알려준다
- ③ append()는 문자열이 아니라 리스트의 메서드다
- ④ 고치기: name += " 3" 또는 리스트에 append()한다

```
(venv) PS C:\robit\python> python main.py  
Traceback (most recent call last):  
  File "main.py", line 2, in <module>  
    name.append(" 3")  
AttributeError: 'str' object has no attribute 'append'
```



에러와 디버깅

-KeyError

딕셔너리에 존재하지 않는 키를 대괄호로 조회할 때 발생한다

```
student = {"name": "민준"}  
print(student["score"])
```



메시지에서 찾을 단서

- ① 마지막 줄의 'score'가 찾지 못한 키다
- ② student.keys()로 실제 키 목록을 확인한다
- ③ 대괄호 조회는 키가 없으면 즉시 오류가 난다
- ④ 고치기: student.get("score", 0) 또는 in으로 확인한다

```
(venv) PS C:\robit\python> python main.py  
Traceback (most recent call last):  
  File "main.py", line 2, in <module>  
    print(student["score"])  
KeyError: 'score'
```

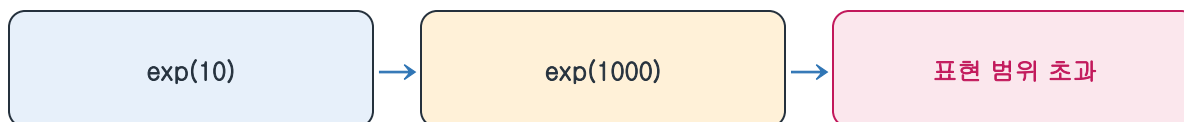


에러와 디버깅

-OverflowError

계산 결과가 사용 중인 수치 표현 범위를 넘어서 처리할 수 없을 때 발생한다

```
import math
result = math.exp(1000)
print(result)
```



메시지에서 찾을 단서

- ① math range error는 수학 함수의 표현 범위 초과를 뜻한다
- ② 파이썬 정수는 커질 수 있지만 실수·C 기반 함수에는 한계가 있다
- ③ 입력값 1000이 지수 함수 결과를 지나치게 크게 만든다
- ④ 고치기: 입력 범위를 제한하거나 로그 값으로 계산한다

```
(venv) PS C:\robit\python> python main.py
Traceback (most recent call last):
  File "main.py", line 2, in <module>
    result = math.exp(1000)
OverflowError: math range error
```



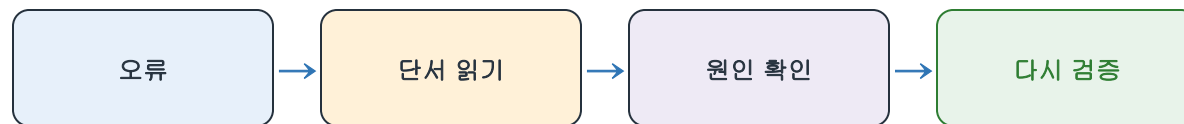
에러와 디버깅

-왜 중요한가

에러 메시지는 프로그램이 멈춘 위치와 원인을 압축해서 알려주는 단서다

증상만 보고 고치기
→ 여러 줄을 한꺼번에 변경
→ 원인을 확인하기 어려움

단서를 읽고 고치기
→ 오류 종류와 줄 번호 확인
→ 한 가지 원인만 검증



디버깅이 필요한 이유

- 같은 오류를 다시 재현할 수 있다
- 원인 범위를 빠르게 좁힐 수 있다
- 수정으로 다른 기능이 깨지는 일을 줄인다
- 해결 과정을 다음 문제에도 재사용할 수 있다

디버깅의 기본 흐름

재현 → 범위 좁히기 → 원인 확인 → 한 가지 수정 → 다시 테스트

에러와 디버깅

-Traceback 읽기

Traceback은 마지막 줄부터 읽고, 필요한 만큼만 위로 올라간다

```
def average(total, count):  
    return total / count  
  
def report():  
    print(average(100, 0))  
  
report()
```

```
1 main.py : 7  
2 report() : 5  
3 average() : 2  
4 ZeroDivisionError
```

읽는 순서

- ① 마지막 줄: ZeroDivisionError와 원인 메시지
- ② 바로 위: 실제로 실패한 코드 줄
- ③ 그 위: 어떤 함수 호출을 거쳐 왔는지
- ④ 내 코드의 가장 가까운 줄부터 확인한다

```
Traceback (most recent call last):  
  File "main.py", line 7, in <module> → report()  
  File "main.py", line 5, in report → average(100, 0)  
  File "main.py", line 2, in average → total / count  
ZeroDivisionError: division by zero
```

에러와 디버깅

-5단계 디버깅

좋은 디버깅은 정답을 맞추는 일이 아니라 문제 범위를 줄이는 과정이다

- 1 같은 입력으로 재현
- 2 기대값과 실제값 비교
- 3 문제 구간을 작게 분리
- 4 한 가지씩 수정
- 5 다시 실행해 검증



도구를 목적에 맞게 사용하기

- `print()`: 값과 실행 순서를 빠르게 확인
- `breakpoint()`: 한 줄씩 실행하며 변수 관찰
- `type()`, `len()`: 자료형과 크기 확인
- `assert`: 반드시 참이어야 하는 조건 기록

디버깅 질문 3개

어떤 입력에서? 기대값과 실제값은? 마지막으로 정상 동작한 지점은?

에러와 디버깅

-값의 흐름 확인

중간값을 출력하면 반복문이 언제부터 잘못되는지 눈으로 확인할 수 있다

```
prices = [1000, 2000, 3000]
total = 0

for price in prices:
    total = price      # 버그
    print(price, total)

# 수정: total += price
```

total = price
결과 3000



total += price
결과 6000

출력에서 발견되는 패턴

- total이 누적되지 않고 매번 현재 price로 바뀐다
- 기대한 마지막 값은 6000, 실제값은 3000이다
- 대입(=)과 누적 대입(+=)의 차이가 원인이다
- breakpoint()로 total 변경 전후를 비교할 수도 있다

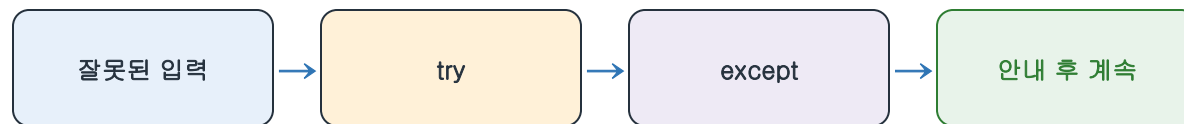
```
> python main.py
1000 1000
2000 2000
3000 3000
기대값 6000 ≠ 실제값 3000
```


예외 처리

-try-except 기본

예상 가능한 입력 실패는 try-except로 처리해 프로그램의 다음 행동을 정한다

```
try:
    age = int(input("나이: "))
    print(f"내년 나이: {age + 1}")
except ValueError:
    print("숫자로 입력해 주세요.")
```



기본 구조

- try: 오류가 날 수 있는 최소한의 코드
- except ValueError: 처리할 오류 종류
- except 안에서는 사용자 안내나 안전한 대체 행동
- 오류 종류를 정확히 좁혀야 진짜 버그가 숨지 않는다

```
> python main.py
```

```
나이: 스무살
```

```
숫자로 입력해 주세요.
```

```
프로그램은 비정상 종료되지 않고 다음 단계로 진행한다
```

예외 처리

-여러 예외 처리

오류 종류별로 다른 안내를 제공하면 사용자가 바로 다음 행동을 선택할 수 있다

```
try:
    with open("score.txt") as file:
        score = int(file.read())
except FileNotFoundError:
    print("score.txt가 없습니다.")
except ValueError:
    print("파일 내용이 숫자가 아닙니다.")
```



분기 기준

- 파일이 없으면 경로를 확인하도록 안내
- 값이 잘못되면 파일 내용을 수정하도록 안내
- 여러 오류를 한 줄에 묶을 수도 있다: `except (A, B):`
- `bare except:`는 `KeyboardInterrupt`까지 숨길 수 있어 피한다

작성 원칙

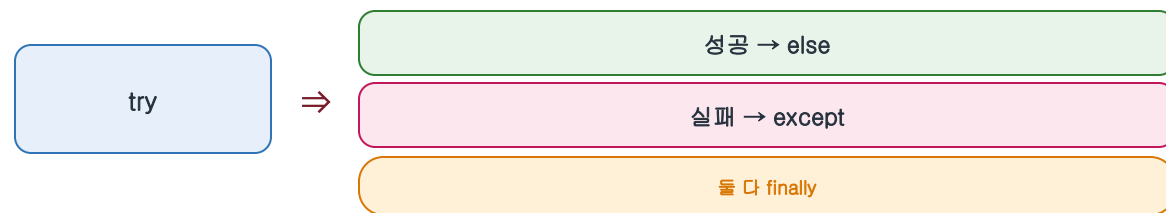
가장 구체적인 예외부터 처리하고, 해결 방법이 같은 경우에만 묶는다

예외 처리

-else와 finally

else는 성공했을 때만, finally는 성공 여부와 관계없이 실행된다

```
try:
    number = int("text")
except ValueError:
    print("변환 실패")
else:
    print(number * 2)
finally:
    print("처리 종료")
```



실행 순서

- 성공: try → else → finally
- 실패: try → except → finally
- else에는 오류가 없어야 실행할 후속 작업을 둔다
- finally에는 파일·연결 정리처럼 반드시 할 일을 둔다

text = '21' → 42 / 처리 종료
text = 'abc' → 변환 실패 / 처리 종료

예외 처리

-예외 객체 활용

예외를 무시하지 말고 원인 메시지와 안전한 반환값을 함께 설계한다

```
def divide(a, b):  
    try:  
        return a / b  
    except ZeroDivisionError as error:  
        print(f"계산 실패: {error}")  
        return None
```



as error가 주는 정보

- 실제 예외 메시지를 로그나 안내문에 활용한다
- 함수는 실패했을 때 None 같은 약속된 값을 돌려준다
- 호출한 쪽은 result is None으로 후속 행동을 정한다
- 해결할 수 없는 예외라면 raise로 다시 전달한다

```
> print(divide(10, 0))
```

```
계산 실패: division by zero
```

```
None
```

오류를 숨긴 것이 아니라 실패를 명시적으로 표현했다

파일 · CSV · JSON

-파일 입출력 기본

with open을 사용하면 작업이 끝날 때 파일이 자동으로 닫힌다

```
with open("memo.txt", "w",  
          encoding="utf-8") as file:  
    file.write("첫 줄\n")  
  
with open("memo.txt", "r",  
          encoding="utf-8") as file:  
    text = file.read()  
  
print(text)
```

r
읽기

w
새로 쓰기

a
이어 쓰기

자주 쓰는 모드

- r: 읽기 — 파일이 없으면 오류
- w: 새로 쓰기 — 기존 내용을 지움
- a: 이어 쓰기 — 기존 내용 뒤에 추가
- encoding='utf-8': 한글 깨짐을 예방

```
> python main.py
```

첫 줄

with 블록을 벗어나면 close()가 자동으로 호출된다

파일 · CSV · JSON

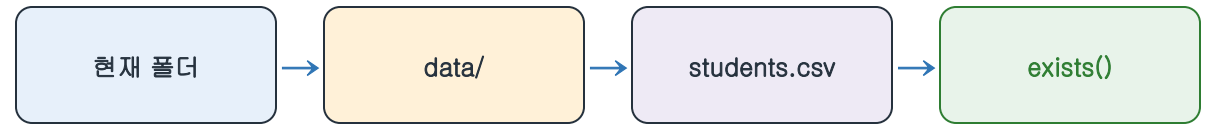
-경로 확인

파일 오류의 대부분은 현재 작업 폴더·상대 경로·인코딩을 확인하면 좁힐 수 있다

```
from pathlib import Path

path = Path("data") / "students.csv"
print(Path.cwd())
print(path.exists())

if path.exists():
    print(path.read_text(encoding="utf-8"))
```



확인 체크리스트

- Path.cwd(): 프로그램이 기준으로 삼는 폴더
- path.exists(): 파일 존재 여부
- resolve(): 절대 경로로 확인
- 파일명·확장자·대소문자·권한·인코딩 확인

상대 경로의 기준

파이썬 파일의 위치가 아니라 프로그램을 실행한 현재 작업 폴더다

파일 · CSV · JSON

-CSV란?

CSV는 행과 열로 된 단순한 표를 쉼표로 구분해 저장하는 텍스트 형식이다

```
name,score
민준,85
서연,92
지우,78
```

name	score
민준	85
서연	92

CSV를 읽을 때 기억할 점

- 첫 행을 열 이름(header)으로 사용하는 경우가 많다
- 한 줄이 한 행, 쉼표 사이가 한 셀이다
- 읽은 값은 기본적으로 문자열이므로 숫자 변환이 필요하다
- 엑셀·스프레드시트·표 형태 데이터 교환에 적합하다

단순한 표·많은 행 → CSV
중첩된 목록과 속성 → JSON

파일 · CSV · JSON

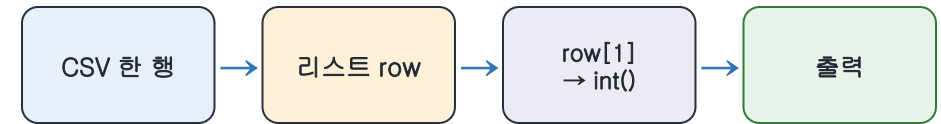
-CSV 읽기: reader

csv.reader는 각 행을 리스트로 읽어 열 번호로 접근한다

```
import csv

with open("students.csv", newline="",
          encoding="utf-8") as file:
    reader = csv.reader(file)
    header = next(reader) # name, score 건너뛰기

    for row in reader:
        name = row[0]
        score = int(row[1])
        print(name, score)
```



핵심 포인트

- row는 ["민준", "85"] 형태의 리스트
- row[0]은 이름, row[1]은 점수
- next(reader)로 첫 행의 열 이름을 건너뛰다
- 모든 값은 문자열이므로 int() 변환이 필요하다

```
> python main.py
민준 85
서연 92
지우 78
```

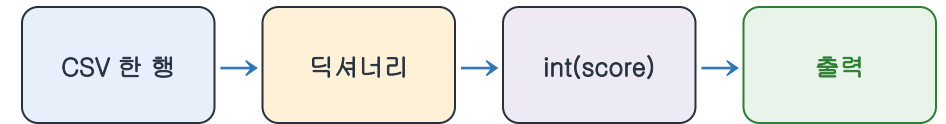

파일 · CSV · JSON

-CSV 읽기

csv.DictReader를 사용하면 열 번호 대신 열 이름으로 값을 읽을 수 있다

```
import csv

with open("students.csv", newline="",
          encoding="utf-8") as file:
    reader = csv.DictReader(file)
    for row in reader:
        name = row["name"]
        score = int(row["score"])
        print(name, score)
```



핵심 포인트

- row는 {'name': '민준', 'score': '85'} 형태의 딕셔너리
- int()로 점수를 숫자로 변환해야 계산할 수 있다
- newline=''은 CSV 줄바꿈 처리를 csv 모듈에 맡긴다
- 파일을 열 때 실제 저장 인코딩과 맞춘다

```
> python main.py
```

```
민준 85
```

```
서연 92
```

```
지우 78
```

파일 · CSV · JSON

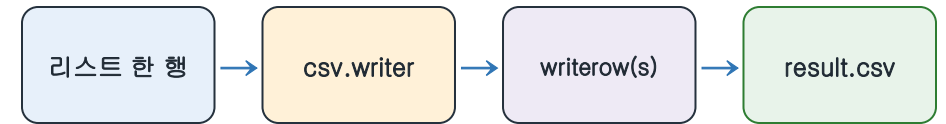
-CSV 쓰기: writer

csv.writer는 리스트·튜플 한 행을 순서대로 CSV에 기록한다

```
import csv

with open("result.csv", "w", newline="",
          encoding="utf-8") as file:
    writer = csv.writer(file)

    writer.writerow(["name", "score"])
    writer.writerow(["민준", 85])
    writer.writerows([
        ["서연", 92],
        ["지우", 78],
    ])
```



쓰는 순서

- ① csv.writer(file)로 작성기를 만든다
- ② writerow(row)는 한 행을 쓴다
- ③ writerows(rows)는 여러 행을 한 번에 쓴다
- ④ 열 순서는 리스트에 넣은 값의 순서와 같다
- ⑤ "w"는 덮어쓰기, "a"는 파일 뒤에 추가한다

```
result.csv
name,score
민준,85
서연,92
지우,78
```

파일 · CSV · JSON

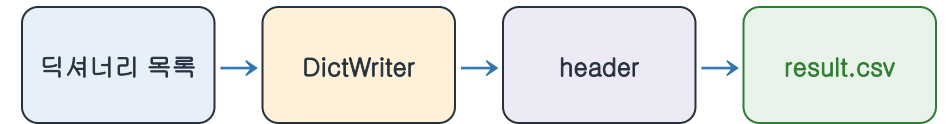
-CSV 쓰기

DictWriter는 열 이름과 딕셔너리 키를 맞춰 일관된 CSV를 만든다

```
import csv

rows = [
    {"name": "민준", "score": 85},
    {"name": "서연", "score": 92},
]

with open("result.csv", "w", newline="",
          encoding="utf-8") as file:
    writer = csv.DictWriter(file, fieldnames=["name", "score"])
    writer.writeheader()
    writer.writerows(rows)
```



쓰는 순서

- ① fieldnames로 열 순서를 정한다
- ② writeheader()로 첫 행에 열 이름을 쓴다
- ③ writerow() 또는 writerows()로 데이터를 쓴다
- ④ w 모드는 기존 파일을 덮어쓰므로 필요한 경우 a를 사용한다

```
result.csv
name,score
민준,85
서연,92
```

CSV는 ‘쉼표’보다 형식 계약이 먼저다

같은 .csv 확장자라도 구분자·따옴표·줄바꿈 규칙은 서로 다를 수 있다

```
# comma + quote
name,note
민준,"서울, 경기 이동"

# semicolon
name;score
민준;85

# quoted newline
민준,"첫째 줄
둘째 줄"
```

설정	무엇을 결정하나	대표값
delimiter	필드를 나누는 문자	, ; Wt
quotechar	구분자·줄바꿈을 포함한 값 감싸기	"
quoting	언제 따옴표를 쓸지	QUOTE_MINIMAL
strict	잘못된 형식을 오류로 볼지	True / False

형식 계약

CSV = 열 구조 + 구분자 + 따옴표 규칙 + 줄바꿈 규칙. 파일명만 보고 규칙을 추측하지 않는다.

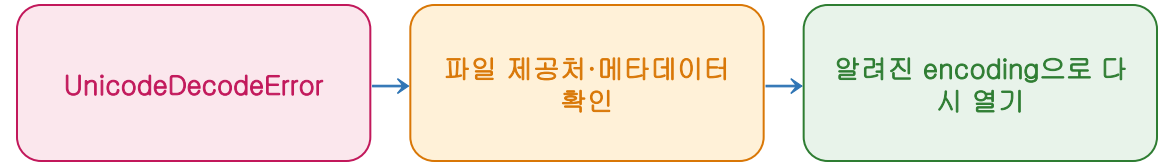
줄바꿈과 인코딩은 반드시 명시한다

내 컴퓨터에서 열린다는 사실보다 다른 환경에서도 같은 행으로 읽히는지 중요하다

```
import csv

with open(
    "students.csv",
    newline="",
    encoding="utf-8",
) as file:
    for row in csv.DictReader(file):
        print(row)

# UTF-8 BOM이 확인된 파일
# encoding="utf-8-sig"
```



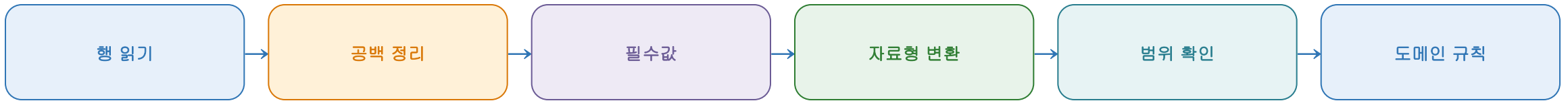
- `newline=""` : csv 모듈이 줄바꿈을 직접 해석한다
- `encoding="utf-8"` : 플랫폼 기본값에 기대지 않는다
- `errors="ignore"` : 손상된 문자를 조용히 버리므로 기본 해결책이 아니다
- 인코딩을 모르면 추측보다 제공처 확인이 먼저다

이식성

`newline=""`와 `encoding`을 코드에 기록하면 운영체제·편집기 차이가 숨은 오류가 되는 것을 줄일 수 있다.

파싱과 검증을 분리하면 원인이 보인다

CSV 문법을 읽는 일과 값이 업무 규칙에 맞는지 판단하는 일은 다른 책임이다



```
raw = {  
    "name": " 민주 ",  
    "score": "105",  
    "email": "",  
}  
  
# 파싱 성공 ≠ 유효한 데이터
```

검증 규칙

- ① name.strip()이 비어 있지 않은가
- ② score가 정수로 변환되는가
- ③ $0 \leq \text{score} \leq 100$ 인가
- ④ 필요한 필드가 모두 있는가

```
def parse_score(raw, line_no):  
    text = raw.strip()  
    if not text:  
        raise ValueError(  
            f"line {line_no}: score empty"  
        )  
    score = int(text)  
    if not 0 <= score <= 100:  
        raise ValueError(  
            f"line {line_no}: {score}"  
        )  
    return score
```

설계 원칙

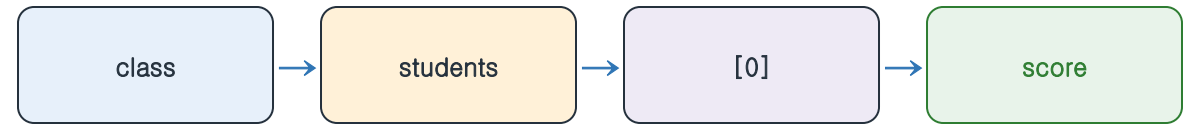
검증 함수는 변환된 값을 반환하고, 실패하면 행 번호·필드명·원래 값을 포함한 구체적 오류를 만든다.

파일 · CSV · JSON

-JSON이란?

JSON은 리스트와 딕셔너리처럼 중첩된 구조를 텍스트로 표현한다

```
{
  "class": "A",
  "students": [
    {"name": "민준", "score": 85},
    {"name": "서연", "score": 92}
  ]
}
```



JSON이 잘 맞는 데이터

- 한 객체 안에 목록이나 다른 객체가 중첩된 구조
- 웹 API의 요청·응답, 설정 파일, 저장 데이터
- 키는 큰따옴표 문자열로 작성한다
- 주석을 지원하지 않으며 문법 오류에 민감하다

Python ↔ JSON

dict ↔ object | list ↔ array | None ↔ null | True ↔ true

파일 · CSV · JSON

-JSON 읽기·쓰기

json.load와 json.dump는 JSON 파일과 파이썬 객체 사이를 변환한다

```
import json

with open("class.json", encoding="utf-8") as file:
    data = json.load(file)

data["students"][0]["score"] = 90

with open("class.json", "w", encoding="utf-8") as file:
    json.dump(data, file,
              ensure_ascii=False, indent=2)
```

JSON 파일
json.load



Python 객체
json.dump

함수 이름 구분

- load / dump: 파일 객체와 변환
- loads / dumps: 문자열과 변환
- ensure_ascii=False: 한글을 그대로 저장
- indent=2: 사람이 읽기 쉬운 들여쓰기

접근 순서

data['students'] → 첫 번째 학생 [0] → 점수 ['score']

JSON은 자료형을 그대로 보존하지 않는다

직렬화는 Python 객체를 공통 표현으로 바꾸는 과정이며 왕복 후 타입이 달라질 수 있다

Python 값	JSON 표현	다시 읽으면
(1, 2)	[1, 2]	list
{1: "A"}	{"1": "A"}	키가 str
datetime(...)	기본 지원 없음	TypeError
Decimal("1.10")	기본 지원 없음	TypeError

```
import json

original = {1: ("A", "B")}
text = json.dumps(original)
restored = json.loads(text)

print(text)
# {"1": ["A", "B"]}

print(restored == original)
# False
```

왕복 계약

JSON이 보존해야 할 것은 Python 타입 자체가 아니라 다른 시스템과 합의한 의미와 필드 구조다.

특수 자료형은 변환 규칙을 직접 정한다

datetime과 Decimal은 문자열·숫자 표현뿐 아니라 복원 규칙까지 함께 설계해야 한다

```
from datetime import datetime
from decimal import Decimal
import json

def to_json(value):
    if isinstance(value, datetime):
        return {
            "type": "datetime",
            "value": value.isoformat(),
        }
    if isinstance(value, Decimal):
        return {
            "type": "decimal",
            "value": str(value),
        }
    raise TypeError(type(value).__name__)

json.dumps(data, default=to_json)
```



복원 규칙도 필요하다

- type 태그 또는 스키마 필드로 의미를 표시한다
- datetime은 시간대 포함 여부를 합의한다
- Decimal은 문자열로 저장하면 자릿수를 보존하기 쉽다
- 알 수 없는 타입은 조용히 바꾸지 말고 TypeError를 낸다

직렬화 경계

default= 함수는 저장 표현을 정할 뿐이다. 다시 읽을 때 원래 의미를 복원하는 규칙은 별도로 필요하다.

정밀도와 호환성은 옵션으로 통제한다

사람이 읽기 좋은 출력과 시스템 간 엄격한 교환은 서로 다른 옵션이 필요하다

한글을 그대로

```
json.dump(  
    data, file,  
    ensure_ascii=False,  
    indent=2,  
)
```

사람이 직접 확인하는 설정 파일·결과 파일에 유용하다.

소수 정밀도 유지

```
from decimal import Decimal  
  
data = json.loads(  
    text,  
    parse_float=Decimal,  
)
```

금액처럼 이진 부동소수점 오차를 피해야 할 값에 사용한다.

표준 밖 숫자 차단

```
json.dumps(  
    data,  
    allow_nan=False,  
)  
  
# NaN / Infinity → ValueError
```

다른 언어·API와 교환할 때 엄격한 JSON 호환성을 높인다.

옵션의 목적

ensure_ascii는 가독성, parse_float는 복원 타입, allow_nan은 표준 호환성을 통제한다.

파일 · CSV · JSON

-CSV와 JSON 비교

파일 형식은 익숙함보다 데이터의 모양과 사용할 작업을 기준으로 선택한다

CSV

행과 열이 일정한 표
엑셀에서 바로 열기 쉬움
많은 행을 순서대로 처리
값은 문자열로 읽힘

데이터 모양



행·열 일정
CSV

중첩 구조
JSON

JSON

- 리스트와 객체가 중첩된 구조
- 웹 API와 설정 파일에 널리 사용
- 숫자·문자열·불리언·null을 구분
- 구조가 유연하지만 표 계산에는 복잡할 수 있다

선택 질문

모든 행의 열이 같은가? → CSV | 데이터 안에 목록·객체가 들어가는가? → JSON

모듈 및 라이브러리

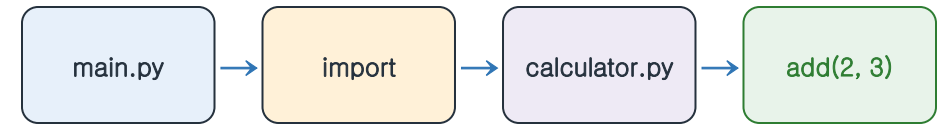
-모듈과 import

모듈은 코드를 파일 단위로 나눠 여러 프로그램에서 다시 사용하게 한다

```
# calculator.py
def add(a, b):
    return a + b

# main.py
from calculator import add

print(add(2, 3))
```



import 방법

- import calculator → calculator.add(2, 3)
- from calculator import add → add(2, 3)
- import numpy as np → 긴 이름을 관례적인 별칭으로 사용
- 파일명은 의미가 분명한 소문자 이름으로 작성한다

```
> python main.py
```

```
5
```

모듈을 직접 실행할 코드에는 `if __name__ == '__main__':`을 활용한다

모듈 및 라이브러리

-표준과 외부 라이브러리

표준 라이브러리는 파이썬에 포함되고, 외부 라이브러리는 별도 설치가 필요하다

```
# 표준 라이브러리
import math
import random
from pathlib import Path

print(math.sqrt(16))
```

표준 라이브러리
바로 import

VS

외부 라이브러리
pip 설치 후 import

외부 라이브러리

- NumPy, pandas, requests처럼 커뮤니티가 배포
- pip로 프로젝트 환경에 설치한 뒤 import
- 버전에 따라 기능이 달라질 수 있어 기록이 필요
- 먼저 표준 라이브러리로 가능한지 확인하면 의존성이 줄어든다

표준: 설치 없이 바로 import

외부: python -m pip install 패키지명 → import

모듈 및 라이브러리

-pip와 가상환경

가상환경은 프로젝트마다 패키지와 버전을 분리해 충돌을 막는다

```
# 1. 가상환경 만들기
python -m venv .venv

# 2. PowerShell에서 활성화
.\.venv\Scripts\Activate.ps1

# 3. 설치와 확인
python -m pip install numpy
python -m pip list

# 4. 버전 기록
python -m pip freeze > requirements.txt
```

프로젝트 A
.venv · NumPy 2

분리

프로젝트 B
.venv · NumPy 1

안전한 사용 습관

- 프롬프트 앞에 (.venv)가 보이는지 확인
- pip 대신 python -m pip를 쓰면 현재 파이썬과 연결이 명확하다
- 프로젝트마다 별도의 .venv를 만든다
- requirements.txt로 같은 환경을 다시 만들 수 있다

작업 순서

만들기 → 활성화 → 설치 → 코드 실행 → 버전 기록

모듈·패키지·프로젝트는 책임이 다르다

파일을 나누는 목적은 길이를 줄이는 것이 아니라 변경 이유와 공개 인터페이스를 분리하는 것이다

```
score_report/  
├── pyproject.toml  
├── src/  
│   └── score_report/  
│       ├── __init__.py  
│       ├── io.py  
│       ├── validation.py  
│       └── summary.py  
└── tests/  
    ├── test_validation.py  
    └── test_summary.py
```

책임 단위

- 모듈(.py): 관련 함수·클래스를 한 파일에 모은다
- 패키지: 여러 모듈을 이름 공간과 공개 API로 묶는다
- 프로젝트: 코드·테스트·메타데이터·의존성을 함께 관리한다

권장 import

```
from score_report.validation import parse_score  
from .summary import average # 패키지 내부 상대 import
```

이름 충돌 방지

프로젝트 파일을 csv.py·json.py·numpy.py처럼 짓지 않는다. 표준·외부 모듈을 가려 import 오류를 만들 수 있다.

재사용 코드와 실행 코드를 분리한다

__name__ 가드를 사용하면 같은 파일을 라이브러리와 실행 프로그램으로 모두 사용할 수 있다

나쁜 예 — import만 해도 실행됨

```
# report.py
def average(values):
    return sum(values) / len(values)

path = input("CSV 경로: ")
print(load_and_summarize(path))

# import report 시 입력 대기 발생
```

좋은 예 — 진입점만 조건부 실행

```
# report.py
def average(values):
    return sum(values) / len(values)

def main():
    path = input("CSV 경로: ")
    print(load_and_summarize(path))

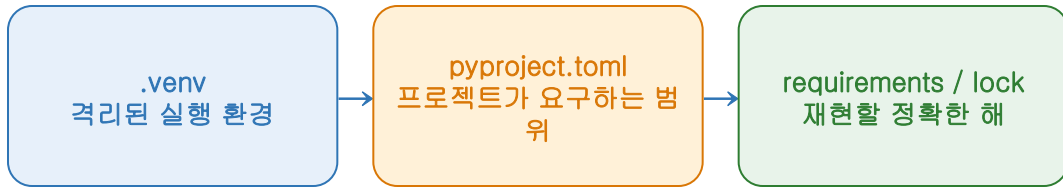
if __name__ == "__main__":
    main()
```

테스트 가능성

함수는 import해 테스트하고, 사용자 입력·파일 선택·출력은 main()에서 조립한다.

설치 목록과 프로젝트 의존성은 다르다

가상환경·pyproject.toml·requirements 파일은 같은 문제를 풀지 않는다



역할을 구분한다

- .venv: 다른 프로젝트와 설치 공간을 분리
- pyproject.toml: 직접 의존성과 지원 범위를 선언
- requirements/lock: 실제 설치 조합을 재현
- .venv 자체는 복사하지 않고 언제든지 다시 만든다

```
# pyproject.toml
[project]
name = "score-report"
dependencies = [
    "numpy>=2,<3",
]

# 환경 만들기
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt

# 현재 환경 점검
python -m pip check
```

재현 절차

새 폴더에서도 “환경 생성 → 의존성 설치 → 테스트 실행”이 성공해야 프로젝트가 재현 가능하다.

버전 범위가 재현성을 결정한다

무조건 최신과 무조건 고정 사이에서 프로젝트 성격에 맞는 제약을 선택한다

상황	예시	의미
버전 미지정	numpy	설치 시점마다 달라질 수 있음
호환 범위	numpy>=2,<3	2.x 업데이트 허용
정확히 고정	numpy==2.4.0	같은 버전 재현
애플리케이션	lock / pinned requirements	검증한 전체 조합 유지

트레이드오프

정확한 고정
재현성 ↑ 보안·기능 업데이트 수동

넓은 범위
업데이트 용이 예기치 않은 변경 위험

검증 명령
python -m pip check
python -m pip freeze

선택 기준

라이브러리는 호환 범위를 선언하고, 배포할 환경은 검증한 조합을 잠그는 전략이 일반적이다.

NumPy 배열 연산

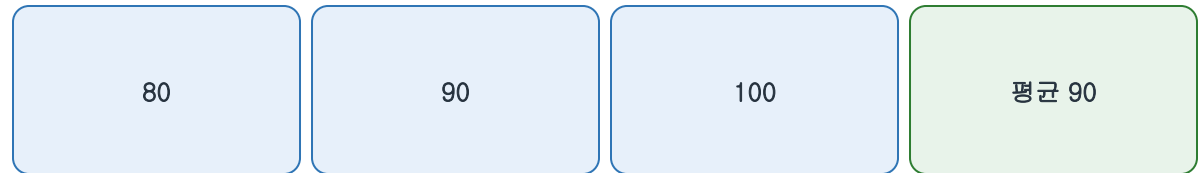
-NumPy란?

NumPy의 ndarray는 많은 숫자를 같은 규칙으로 빠르고 간결하게 계산한다

```
import numpy as np

scores = np.array([80, 90, 100])

print(scores)
print(scores.mean())
print(scores.shape, scores.dtype)
```



ndarray의 특징

- 같은 종류의 값을 연속된 배열로 다룬다
- 반복문 없이 배열 전체에 연산을 적용한다
- shape는 배열의 모양, dtype은 값의 자료형이다
- 과학 계산·데이터 분석 라이브러리의 기초가 된다

```
> python main.py
[ 80  90 100]
90.0
(3,) int64
```

NumPy 배열 연산

-리스트와 배열의 차이

리스트의 +는 이어 붙이기이고, NumPy 배열의 +는 원소별 계산이다

```
list_scores = [1, 2, 3]
print(list_scores + [10])
```

```
# [1, 2, 3, 10]
# 하나의 원소를 뒤에 추가
```

리스트 + [10]
[1, 2, 3, 10]

≠

배열 + 10
[11, 12, 13]

NumPy 배열

```
import numpy as np
array_scores = np.array([1, 2, 3])
print(array_scores + 10)
```

결과: [11 12 13] — 모든 원소에 10을 더한다

핵심 차이

리스트는 범용 자료 구조 | NumPy 배열은 수치 계산 규칙이 내장된 자료 구조

NumPy 배열 연산

-배열 연산

브로드캐스팅과 불리언 인덱싱으로 반복문 없이 전체 데이터를 변환·선택할 수 있다

```
import numpy as np

scores = np.array([72, 88, 95, 64])

passed = scores >= 80
print(passed)
print(scores[passed])

bonus = scores * 1.1
print(bonus)
```



두 가지 핵심 연산

- `scores >= 80` → 원소마다 True/False 비교
- `scores[passed]` → True인 원소만 선택
- `scores * 1.1` → 모든 원소에 같은 계산 적용
- 모양이 맞는 배열끼리는 원소별 `+`, `-`, `*`, `/`가 가능하다

```
[False  True  True False]
[88 95]
[ 79.2  96.8 104.5  70.4]
```

NumPy 배열 연산

-요약 통계

평균·최댓값·표준편차를 조합하면 데이터의 중심과 퍼짐을 빠르게 파악할 수 있다

```
import numpy as np

scores = np.array([85, 92, 78, 90, 88])

print(scores.mean())
print(scores.max())
print(scores.std().round(1))
```



결과 해석

- mean(): 평균 86.6 — 전체 수준
- max(): 92 — 가장 큰 값
- std(): 4.9 — 평균 주변에 퍼진 정도
- 2차원 배열에서는 axis로 행 또는 열 기준을 정한다

```
> python main.py
86.6
92
4.9
```

과제1 | CSV 성적 정리 프로그램

-CSV·JSON 종합 과제

잘못된 행을 건너뛰고 유효한 성적만 CSV와 JSON으로 저장한다

students.csv

name,score

민준,85

서연,92

지우,abc

하늘,105

유진,78

abc → 숫자 변환 실패

105 → 허용 범위 초과

students.csv

csv.reader

int +
범위 검사

csv.writer

clean.csv

json.dump
summary.json

필수 기능

- 1 첫 행을 header로 건너뛴다
- 2 score를 int로 바꾸고 0~100 범위를 검사한다
- 3 오류 행은 이유를 출력하고 다음 행을 계속 처리한다
- 4 유효 행은 clean_students.csv로 저장한다
- 5 인원수·평균·최고점을 summary.json으로 저장한다

제출 main.py · clean_students.csv · summary.json

완료 reader / writer 사용 · FileNotFoundError / ValueError 처리

추가 구현 같은 기능을 DictReader / DictWriter로 다시 구현

과제2 | 외부 라이브러리 사용

-임의의 파이썬 라이브러리를 선택해서 설치한 뒤 해당 라이브러리의 기능을 적용한 코드 작성

조건

- 1 pip install로 외부 라이브러리 설치
- 2 설치된 라이브러리의 버전 확인 및 명시
- 3 라이브러리를 활용한 main.py 코드 작성
- 4 pip freeze를 통한 requirements.txt 작성
- 5 실행 방법 및 코드 설명, 라이브러리의 이해에 대한 보고서 작성 (README 파일로 작성)

README 파일이란? - 깃허브 레포지토리 설명 등을 적어놓는 파일로 프로젝트의 전반적인 설명, 프로젝트 정보, 라이선스 정보, 외부 리소스 · 라이브러리 정보, 프로젝트 코드 설치 및 실행 방법 등을 적어 놓아 내 프로젝트를 다른 사람이 처음 보아도 쉽게 이해할 수 있도록 적어놓는 파일.

README.md 파일로 작성 권장. 만약 markdown 형식의 포맷이 익숙치 않다면 .txt 포맷의 README도 무방.

예시 : https://github.com/Ericksii/FAST_LIO_ROS2

제출 main.py · requirements.txt · README

추가 구현 라이브러리 선택 및 버전 선택에 대한 이유 등 과제의 기본 틀 안에서의 자유로운 확장 가능

과제 제출

과제 제출 형식:

- 과제 1~4, 고급과제 1은 (문제 번호.py)로 제출
- 고급과제 2는 보고서(pdf) 및 .py파일 2개 제출

제출 형태 자유

- 깃허브 등록 후 깃허브 주소
- 파일 압축 후 메일 제출 (kwrobitcontact@gmail.com)
- 1일차 과제도 이전 메일주소로만 보내놓았을 경우 위 메일로 다시 제출

모든 코드에는 최소한의 주석 설명 작성

기한

수요일 출근 전(1,2일차 모두 제출)

Thank you
